

Curso Avanzado de Ollama con Python: Guía Completa de Implementación

Contenidos del Curso

1. Introducción al Ecosistema Ollama	2
1.1. ¿Por qué Ollama? Ventajas del despliegue local frente a APIs en la nube.....	2
1.2. Arquitectura de Ollama: Servidor, API REST y el Wrapper de Python.....	2
1.3. Instalación avanzada: Docker vs Nativo.	2
1.4. Configuración de variables de entorno (OLLAMA_HOST, OLLAMA_MODELS).	3
1.5. Parámetros de Configuración Avanzada.	3
2. Dominando la Librería ollama-python	4
2.1. Instalación y Gestión de Entornos.....	4
2.2. Funciones Globales vs. Client vs. AsyncClient.	4
2.3. Sesión Práctica: Inspección de Capacidades del Hardware.....	5
3. Gestión Completa de Modelos (CRUD).....	6
3.1. Listado y Estado del Servidor.	6
3.2. Descarga (Pull) y Actualización.	6
3.3. Creación de Modelos Personalizados (Create).	7
3.4. Copia (Copy) y Renombrado.	10
3.5. Eliminación (Delete).	11
3.6. Publicación (Push).....	12
3.7. Verificación de Detalles (Show).	12
4. Inferencia y Generación de Texto Profunda	13
4.1. Generate API: Inferencia Atómica y Estructurada.	13
4.2. Chat API: Gestión de Contexto y Memoria de Sesión.	14
4.3. Streaming: Optimizando la Latencia Percibida.	15
5. Tools y Function Calling	15
5.1. Tools: Extensión de Capacidades con Funciones Personalizadas.	15
5.2. Extracción de Datos de las respuestas de inferencia.	17
5.3. Conceptos de Agentes y herramientas externas.	18
5.4. Ejemplo Práctico: Agente de Información Meteorológica.....	20
6. Visión y Modelos Multimodales	21
6.1. Capacidades de visión (OCR, descripción, análisis).....	21
7. Embeddings y Flujos RAG (Retrieval Augmented Generation).....	22
7.1. Fundamentos de Embeddings: Del Texto al Espacio Vectorial.....	22
7.2. Arquitectura RAG: Combinar Memoria y Recuperación para Respuestas Actualizadas.	23

7.3. Ejemplo Funcional: Sistema de Soporte con Datos Propios.	23
8. Escalabilidad y Asincronía.	24
8.1. Programación no bloqueante con Asyncio.	24
8.2. Escalado Horizontal: Múltiples Instancias de Ollama.	25
8.3. Monitorización y Logging en Ollama.	26
9. Mejores Prácticas y Rendimiento.	27
9.1. Manejo de Errores y Robustez del Sistema.	27
9.2. Optimización de Recursos y Cuantización.	28
10. Proyecto Final: Agente de Soporte Técnico Autónomo.	29

1. Introducción al Ecosistema Ollama

1.1. ¿Por qué Ollama? Ventajas del despliegue local frente a APIs en la nube.

Ollama permite ejecutar modelos de lenguaje de gran tamaño (LLM) en tu propia máquina.

Las ventajas clave son:

- **Privacidad:** Tus datos nunca salen de tu infraestructura local. Ideal para manejar información sensible o corporativa.
- **Coste:** Sin cuotas de suscripción ni pagos por tokens consumidos. El coste se limita al hardware y electricidad.
- **Latencia:** Menor tiempo de respuesta al no depender de servidores externos ni de la velocidad de tu conexión a internet.
- **Soberanía:** Tienes el control total sobre qué modelos usas, cuándo los actualizas y cómo los configuras sin depender de cambios de políticas de terceros.

1.2. Arquitectura de Ollama: Servidor, API REST y el Wrapper de Python.

Ollama se divide en dos componentes principales:

1. **Ollama Serve:** Un servicio en segundo plano (daemon) que gestiona la carga de modelos en la GPU/RAM utilizando bibliotecas como llama.cpp y expone una API REST en el puerto 11434.
2. **Librería Python:** Un cliente ligero que abstrae las llamadas HTTP a la API, facilitando el uso de objetos Python y tipos de datos nativos en lugar de manejar JSON crudo y cabeceras HTTP manualmente.

1.3. Instalación avanzada: Docker vs Nativo.

- **Nativo:** Se instala directamente en el sistema operativo. Es la opción recomendada para exprimir al máximo la GPU (CUDA en Linux/Windows o Metal en Apple Silicon).

- **Docker:** Ideal para entornos aislados o despliegues tipo servidor. Permite levantar Ollama junto a interfaces gráficas como *Open WebUI* o *Automatic1111* de forma rápida y reproducible.

1.4. Configuración de variables de entorno (OLLAMA_HOST, OLLAMA_MODELS).

Puedes personalizar el servidor mediante variables de entorno:

- **OLLAMA_HOST:** Define la dirección y el puerto (ej: `0.0.0.0:11434` para permitir acceso desde otras máquinas de la red).
- **OLLAMA_MODELS:** Ruta personalizada para guardar los modelos (fundamental si tienes un SSD pequeño y quieres mover los modelos a un disco mecánico o externo).
- **OLLAMA_KEEP_ALIVE:** Controla cuánto tiempo permanece el modelo en memoria tras la última petición (por defecto 5m).

1.5. Parámetros de Configuración Avanzada.

Para optimizar el rendimiento y el comportamiento de los modelos, Ollama permite ajustar parámetros críticos a través de la API o del `Modelfile`. Estos parámetros determinan desde la creatividad del modelo hasta el uso de recursos de hardware.

Conceptos clave:

- **Determinismo vs. Creatividad:** La temperatura controla la aleatoriedad. Un valor de `0` siempre dará la misma respuesta para el mismo prompt (ideal para código o extracción de datos), mientras que `0.8` o superior fomenta la variedad (ideal para escritura creativa).
- **Ventana de Contexto (`num_ctx`):** Es la "memoria de corto plazo". Si es muy baja, el modelo olvidará el principio de una conversación larga. Si es muy alta, consumirá mucha VRAM.
- **Penalización de Repetición:** Evita que el modelo entre en bucles infinitos repitiendo la misma frase.

```
import ollama

# Ejemplo de configuración de parámetros avanzados en una petición
response = ollama.generate(
    model='llama3.2',
    prompt='Explica qué es un agujero negro a un niño de 5 años.',
    options={
        'num_ctx': 4096,      # Tamaño de la ventana de contexto
        'temperature': 0.7,   # Balance entre creatividad y coherencia
        'top_k': 40,          # Limita el vocabulario a las K palabras más probables
        'top_p': 0.9,         # Núcleo de muestreo (Nucleus sampling)
        'num_gpu': 1,         # Forzar el uso de la GPU (0 para CPU)
        'repeat_penalty': 1.1 # Penaliza la repetición de palabras
    }
)
```

```
print(f"Respuesta con parámetros optimizados:\n{response['response']}")
```

Tabla de Parámetros Clave:

Parámetro	Descripción	Valor Típico
<code>num_ctx</code>	Tamaño del contexto (tokens).	2048 - 8192
<code>temperature</code>	Aleatoriedad de la respuesta.	0.0 - 1.0
<code>top_k</code>	Limita el vocabulario a las K palabras más probables.	10 - 100
<code>top_p</code>	Núcleo de muestreo (Nucleus sampling).	0.8 - 0.95
<code>num_predict</code>	Máximo de tokens a generar.	128 - 2048
<code>stop</code>	Secuencias que detienen la generación.	<code>["\n", "User:"]</code>

2. Dominando la Librería ollama-python

2.1. Instalación y Gestión de Entornos.

Para trabajar con Ollama en Python, se recomienda el uso de entornos virtuales (**venv**) para evitar conflictos de dependencias. La librería es ligera y solo requiere **httpx** como dependencia base.

```
# Creación de entorno virtual
python -m venv venv
source venv/bin/activate # En Windows: venv\Scripts\activate

# Instalación de la librería oficial
pip install ollama
```

2.2. Funciones Globales vs. Client vs. AsyncClient.

La librería ofrece tres niveles de abstracción según la complejidad del proyecto:

- Funciones Globales:** Acceso rápido al servidor por defecto (**localhost:11434**).
- Client (Síncrono):** Recomendado para scripts, herramientas de CLI y aplicaciones donde se gestionan múltiples servidores. Permite configurar timeouts y cabeceras personalizadas.
- AsyncClient (Asincrónico):** Orientado a aplicaciones de alto rendimiento y servidores web.

```
import ollama
from ollama import Client

# Ejemplo con Cliente Síncrono configurado
# Ideal si Ollama corre en un servidor externo o en Docker con puerto mapeado
client = Client(host='http://192.168.1.50:11434')
```

```
try:
    response = client.list()
    print("Conexión exitosa al servidor remoto.")
except Exception as e:
    print(f"Error de conexión: {e}")
```

2.3. Sesión Práctica: Inspección de Capacidades del Hardware.

Antes de cargar un modelo, debemos entender los límites de nuestra máquina. Un modelo de 7B parámetros requiere aproximadamente 4-5 GB de VRAM libre.

Conceptos clave:

- **Model Size:** Tamaño en disco.
- **VRAM/RAM:** Si el modelo no cabe en la GPU, Ollama lo moverá a la RAM (sistema "offloading"), lo que ralentizará la generación significativamente.

```
import ollama

def inspeccionar_sistema():
    print("--- Análisis de biblioteca local ---")
    models_info = ollama.list()

    for model in models_info['models']:
        name = model['name']
        size_gb = model['size'] / (1024**3)
        # Extraemos la cuantización de los metadatos
        details = model.get('details', {})
        quant = details.get('quantization_level', 'Desconocida')

        print(f"Modelo: {name:<20} | Tamaño: {size_gb:>5.2f} GB | Cuantización: {quant}")

    # Verificación de carga actual
    print("\n--- Estado de la memoria activa (Ollama PS) ---")
    active = ollama.ps()
    if not active['models']:
        print("No hay modelos cargados en memoria actualmente.")
    else:
        for m in active['models']:
            print(f"En ejecución: {m['name']} (Ocupando VRAM)")

inspeccionar_sistema()
```

3. Gestión Completa de Modelos (CRUD)

3.1. Listado y Estado del Servidor.

Antes de operar, debemos conocer qué modelos tenemos y cuáles están consumiendo recursos. La gestión de modelos en Ollama no se limita a saber qué hay en el disco; implica entender qué modelos están cargados en la VRAM (tarjeta gráfica) o en la RAM del sistema.

Conceptos clave:

- **list() vs ps():** `list()` devuelve el inventario físico (archivos `.bin` o `.gguf` en disco). `ps()` devuelve el estado de ejecución (qué modelos están "calientes" en memoria).
- **Cuantización:** Verás niveles como `Q4_K_M`. Esto indica que el modelo ha sido comprimido para ocupar menos espacio y memoria, sacrificando una fracción mínima de precisión. Es vital para correr modelos grandes en GPUs de consumo.
- **Tamaño en Disco vs. VRAM:** Un modelo de 5GB en disco suele ocupar aproximadamente lo mismo en VRAM, pero Ollama puede añadir un margen para el procesamiento del contexto.

```
import ollama
from datetime import datetime

def format_size(bytes):
    return f"{bytes / (1024**3):.2f} GB"

# 1. Listado de inventario local (Modelos descargados)
print("=== Inventario de Modelos en Disco ===")
models = ollama.list()
for model in models['models']:
    # Formateamos la fecha de modificación si está disponible
    modified = model.get('modified_at', 'Desconocida')
    print(f"- {model['name']:20} | Tamaño: {format_size(model['size'])} | Modificado: {modified}")

# 2. Estado de ejecución (Modelos cargados en memoria)
print("\n=== Modelos Activos en Memoria (VRAM/RAM) ===")
active = ollama.ps()
if not active['models']:
    print("El servidor está en hibernación. No hay modelos cargados.")
else:
    for m in active['models']:
        print(f"-> {m['name']:20} | Ocupación: {format_size(m['size'])}")
        print(f"    Expira en: {m['expires_at']}") # Cuánto tiempo le queda antes de
        descargarse
```

3.2. Descarga (Pull) y Actualización.

Podemos automatizar la descarga de modelos antes de ejecutar una tarea crítica, asegurándonos de

que están disponibles localmente. Ollama permite el "streaming" de la descarga para poder mostrar una barra de progreso al usuario, lo cual es fundamental para modelos que pueden pesar varios gigabytes.

Conceptos clave:

- **Inmutable vs. Dinámico:** Los modelos se identifican por su nombre (`llama3:8b`). Si pides un modelo sin tag (`llama3`), Ollama bajará la versión marcada como `latest`.
- **Diferencial (Delta Pull):** Si intentas descargar un modelo que ya tienes pero ha sido actualizado en la biblioteca central, Ollama solo descargará las capas (layers) que han cambiado, ahorrando ancho de banda.
- **Seguimiento del 'status':** Durante la descarga, el servidor emite diferentes estados: `pulling`, `manifest`, `downloading`, `verifying` y `completed`.

```
import sys
import ollama

def descargar_con_progreso(nombre_modelo):
    print(f"Iniciando descarga de {nombre_modelo}...")

    # Activamos stream=True para recibir actualizaciones en tiempo real
    stream = ollama.pull(nombre_modelo, stream=True)

    for item in stream:
        status = item.get('status', '')
        completed = item.get('completed', 0)
        total = item.get('total', 1) # Evitamos división por 0

        # Calculamos porcentaje si tenemos el total
        if total > 0:
            porcentaje = (completed / total) * 100
            sys.stdout.write(f"\r[{{status}}] {{porcentaje:5.1f}}% completado...")
            sys.stdout.flush()
        else:
            sys.stdout.write(f"\r[{{status}}] Trabajando...")
            sys.stdout.flush()

    print(f"\n¡Descarga de {nombre_modelo} finalizada con éxito!")

# Uso:
# descargar_con_progreso('mistral:7b')
```

3.3. Creación de Modelos Personalizados (Create).

Mediante un `Modelfile`, podemos definir el comportamiento base, parámetros (temperatura, contexto) y el prompt de sistema. Esta es la forma técnica de crear "agentes" con personalidades predefinidas.

Conceptos clave:

- **FROM:** La base. Puede ser un modelo ya existente (**llama3**) o un archivo **.gguf** directo.
- **SYSTEM:** Define el comportamiento, tono y limitaciones. Es lo primero que lee el modelo.
- **PARAMETER:** Configura ajustes técnicos (**temperature**, **num_ctx**, **repeat_penalty**).
- **TEMPLATE:** Define cómo se estructuran los mensajes (**USER:**, **ASSISTANT:**). Normalmente ya viene bien por defecto con Ollama.

Los parámetros del modelfile son:

Para ajustar el comportamiento del modelo en un **Modelfile**, se usan instrucciones **PARAMETER**. En la práctica, Ollama acepta enteros, flotantes o cadenas según el parámetro. Cuando la documentación no fija un máximo estricto, la tabla indica un **rango operativo de referencia** y no un límite duro del motor.

Parámetro	Valores válidos en Ollama	Qué controla	Rango de uso recomendado	Valor por defecto	Valor de referencia
mirostat	0, 1, 2	Activa el muestreo Mirostat para controlar dinámicamente la entropía de la generación.	0 si quieres un comportamiento predecible; 1 o 2 si buscas más control adaptativo.	0	0
mirostat_eta	flotante > 0	Velocidad de ajuste de Mirostat ante la retroalimentación del texto generado.	0.05 - 0.2	0.1	0.1
mirostat_tau	flotante > 0	Objetivo de entropía de Mirostat; afecta al equilibrio entre foco y diversidad.	3.0 - 8.0	5.0	5.0
num_ctx	entero > 0	Tamaño de la ventana de contexto disponible para la inferencia.	2048 - 8192 según RAM/VRAM; más alto consume más memoria.	2048	4096
repeat_last_n	entero; 0 desactiva, -1 usa num_ctx	Cuántos tokens hacia atrás revisa el modelo para penalizar repeticiones.	32 - 256	64	64
repeat_penalty	flotante > 0	Intensidad de la penalización por repetición.	1.05 - 1.3	1.1	1.1

Parámetro	Valores válidos en Ollama	Qué controla	Rango de uso recomendado	Valor por defecto	Valor de referencia
temperature	flotante ≥ 0	Aleatoriedad global de la generación.	0.0 - 1.0 para la mayoría de usos; 1.2+ solo para escritura creativa.	0.8	0.7
seed	entero	Semilla aleatoria para reproducibilidad.	cualquier entero fijo si quieres respuestas repetibles.	0	42
stop	cadena; se puede declarar varias veces	Secuencias de parada que fuerzan el fin de la generación.	1 - 4 secuencias bien definidas según tu plantilla.	sin valor por defecto explícito	"Usuario:" o tokens especiales del template
num_predict	entero; -1 para generación ilimitada	Máximo de tokens a generar por respuesta.	128 - 1024 en chat; 1024 - 4096 en redacción larga.	-1	512
top_k	entero positivo	Limita la selección a las k palabras más probables.	20 - 100	40	40
top_p	flotante entre 0 y 1	Muestreo nucleus; conserva los tokens cuya probabilidad acumulada entra en el umbral.	0.8 - 0.95	0.9	0.9
min_p	flotante entre 0 y 1	Filtra tokens con probabilidad demasiado baja respecto al token más probable.	0.02 - 0.1	0.0	0.05
tfs_z	flotante ≥ 1 ; 1 equivale a desactivado	Tail Free Sampling; reduce el peso de tokens poco probables.	1.0 - 2.0	1.0	1.0

Notas importantes:

- temperature, top_k, top_p, min_p, tfs_z y mirostat* interactúan entre sí. Conviene cambiar uno o dos parámetros cada vez para no perder trazabilidad.
- num_ctx y num_predict son los parámetros con mayor impacto en consumo de memoria y latencia.
- stop no tiene un rango numérico: su validez depende de que la secuencia exista realmente en tu TEMPLATE o en el formato conversacional del modelo.

- En algunas versiones antiguas de la documentación puede aparecer `num_predict = -2` como modo de relleno del contexto. Si necesitas compatibilidad estricta, conviene verificar la versión instalada con `ollama --version`.

¿Por qué crear modelos personalizados?

1. **Consistencia:** Aseguramos que siempre use los mismos parámetros de temperatura y sistema.
2. **Seguridad:** Podemos prohibir temas específicos en el `SYSTEM PROMPT`.
3. **Facilidad de Uso:** En lugar de enviar un prompt gigante cada vez, simplemente cargamos el modelo `expert-chef`.

```
import ollama

# 1. Definimos el Modelfile como una cadena multilínea
# 'expert-chef' será un experto en cocina que solo usa ingredientes comunes.
def crear_chef_modelo():
    modelfile = """
    FROM llama3
    PARAMETER temperature 0.5
    PARAMETER num_ctx 4096
    SYSTEM "Eres un chef experto. Tus consejos culinarios deben ser rápidos,
            económicos y centrados en ingredientes que se encuentran en España.
            No menciones ingredientes exóticos o caros a menos que te pregunten."
    """

    print("Creando modelo 'expert-chef'...")
    # ollama.create registrará el modelo en la base de datos local
    ollama.create(model='expert-chef', modelfile=modelfile)
    print("¡Modelo creado y listo para usar!")

# Uso:
# crear_chef_modelo()
# res = ollama.generate(model='expert-chef', prompt='¿Qué puedo hacer con 3 huevos y tomate?')
# print(res['response'])
```

3.4. Copia (Copy) y Renombrado.

La función `copy` de Ollama permite crear alias o variantes de un mismo modelo sin duplicar el peso en el disco (gracias al sistema de deduplicación de capas de Ollama). No existe la función "rename" como tal, por lo que usaremos una estrategia combinando Copy y Delete.

Conceptos clave:

- **Inexistencia de Duplicación Real:** Copiar un modelo de 5GB de `llama3` a `mi-llama-copia` no consumirá 5GB más de espacio. Solo se crearán punteros a los mismos archivos binarios.
- **Estrategia Rename:** Para "cambiar de nombre", lo copiamos al destino y borramos el origen. Es una operación segura pero que debemos manejar con cuidado en producción si hay procesos en medio.

- **Versionado con Alias:** Es útil para crear puntos de control (**checkpoint**) antes de experimentar con un nuevo **Modelfile** sobre un modelo personalizado.

```
import ollama

def gestionar_alias(nombre_original, nuevo_nombre):
    print(f"Copiando {nombre_original} -> {nuevo_nombre}...")
    try:
        # 1. Copiamos para asegurar el destino
        ollama.copy(nombre_original, nuevo_nombre)
        print("Copia realizada con éxito.")

        # 2. Borramos el original para simular el 'rename' si es necesario
        # Descomenta bajo tu responsabilidad:
        # ollama.delete(nombre_original)
        # print(f"Modelo {nombre_original} eliminado, renombramiento completado.")

    except Exception as e:
        print(f"Ocurrió un error en la gestión: {e}")

# Ejemplo de alias:
# gestionar_alias('llama3:latest', 'llama3:produccion')
```

3.5. Eliminación (Delete).

Gestión de espacio en disco eliminando modelos que ya no se utilizan. Es fundamental para mantener el servidor limpio, sobre todo si experimentas con muchas variantes o descargas modelos pesados.

Conceptos clave:

- **Fallo Silencioso vs Errores:** Al borrar un modelo que no existe, **ollama-python** lanzará una excepción. Es buena práctica usar un bloque **try-except** o listar antes de borrar.
- **Liberación de Memoria:** Borrar un modelo en disco **no** lo descarga de la VRAM inmediatamente si se está usando. Es preferible llamar a borrarlos cuando no hay peticiones activas.

```
import ollama

def limpiar_modelos(lista_modelos):
    modelos_actuales = [m['name'] for m in ollama.list()['models']]

    for nombre in lista_modelos:
        if nombre in modelos_actuales:
            print(f"Borrando modelo de prueba: {nombre}...")
            ollama.delete(nombre)
        else:
            print(f"Aviso: El modelo {nombre} ya no existe o ya fue borrado.")
```

```
# Uso:
# limpiar_modelos(['temp-model-01', 'temp-model-02'])
```

3.6. Publicación (Push).

Si tienes una cuenta en ollama.com, puedes subir tus modelos personalizados o versiones tuneadas para compartirlas con la comunidad o desplegarlas fácilmente en otros servidores.

Conceptos clave:

- **Formato de 'Tagging':** El nombre debe seguir siempre el patrón `nombre_usuario/nombre_modelo:etiqueta`.
- **Pre-requisito CLI:** Para que el comando funcione vía Python, primero debes haber configurado las llaves SSH y el login en la terminal local (`ollama login`).
- **Capas (Layers):** Al igual que en Docker, Ollama solo enviará al servidor las capas (layers) que no existen ya en el repositorio central.

```
import ollama

def publicar_modelo(mi_usuario, nombre_local):
    modelo_remoto = f"{mi_usuario}/{nombre_local}:v1"

    # Pre-requisito: Copiar el modelo local al nombre con tag de usuario
    ollama.copy(nombre_local, modelo_remoto)

    print(f"Subiendo {modelo_remoto} a la biblioteca central...")

    # Al igual que el Pull, el Push permite streaming del progreso
    stream = ollama.push(modelo_remoto, stream=True)
    for p in stream:
        print(f"\rEstado subir: {p.get('status')}\n...", end='')

# Ejemplo (Suponiendo que ya hiciste login en CLI):
# publicar_modelo('mi_git_user', 'expert-chef')
```

3.7. Verificación de Detalles (Show).

Obtener información técnica detallada: arquitectura, parámetros de cuantización, licencia y esquema del sistema. La función `show` permite inspeccionar la "caja negra" de un modelo. Nos proporciona metadatos técnicos fundamentales sobre cómo está construido y qué instrucciones base tiene.

```
import ollama
```

```
def inspeccion_tecnica(modelo):
    info = ollama.show(modelo)

    print(f"=== Detalles de {modelo} ===")
    print(f"Arquitectura: {info['details']['family']}")
    print(f"Cuantización: {info['details'].get('quantization_level', 'N/A')}")
    print(f"Parámetros del Modelfile:\n{info.get('parameters', 'Ninguno')}")
    print(f"Sistema (System Prompt):\n{info.get('system', 'Sin definir')}")

# Uso:
# inspeccion_tecnica('llama3')
```

4. Inferencia y Generación de Texto Profunda

4.1. Generate API: Inferencia Atómica y Estructurada.

La función **generate** es la unidad mínima de inferencia. A diferencia de **chat**, no mantiene un historial automático, lo que la hace ideal para tareas de procesamiento de lenguaje natural (NLP) puras.

Conceptos clave:

- **Prompt Engineering:** La calidad del resultado depende del contexto.
- **Formato de Salida:** Podemos forzar al modelo a responder en formatos específicos como JSON para facilitar la integración con otras aplicaciones.

```
import ollama

# Generación estructurada (JSON)
# Ideal para extraer entidades o clasificar textos
prompt_extraccion = """
Extrae el nombre del producto y el precio del siguiente texto:
'Me compré unas zapatillas Nike a 89 euros ayer.'
Responde exclusivamente en formato JSON.
"""

response = ollama.generate(
    model='llama3',
    prompt=prompt_extraccion,
    format='json', # Forzamos esquema JSON
    options={
        'temperature': 0.0, # 0.0 para máxima consistencia y mínima creatividad
        'num_predict': 50,
    }
)
print("Resultado Estructurado:")
```

```
print(response['response'])
```

4.2. Chat API: Gestión de Contexto y Memoria de Sesión.

Para aplicaciones interactivas, **chat** es la herramienta adecuada. La clave aquí es entender que el LLM es "stateless" (sin estado); cada vez que llamamos a la API, debemos enviar **todo el historial** para que el modelo "recuerde" lo dicho anteriormente.

Para construir aplicaciones de chat efectivas, es fundamental entender cómo Ollama interpreta los diferentes roles en la lista de mensajes. Aunque el modelo sea técnicamente "stateless" (sin estado), la estructura de roles permite simular una conversación coherente.

Roles Clave:

- **system (El Contexto Operativo)**: Define la personalidad, límites y conocimientos base del modelo. Se envía al principio y condiciona el resto de la conversación. Es el "ADN" del asistente.
- **user (La Intención Humana)**: Contiene las preguntas o comandos directos del usuario.
- **assistant (La Memoria de Respuesta)**: Representa las respuestas anteriores generadas por la IA. Incluirlas en el historial es lo que permite que el modelo tenga "memoria".

```
import ollama

# Ejemplo de estructura de mensajes con roles
mensajes = [
    {
        'role': 'system',
        'content': 'Eres un tutor de programación experto en Python. Responde de forma breve y fomenta que el estudiante piense.'
    },
    {
        'role': 'user',
        'content': '¿Cómo puedo invertir una lista en Python?'
    },
    {
        'role': 'assistant',
        'content': 'Hay varias formas de hacerlo. ¿Has oído hablar del slicing o del método .reverse()?'
    },
    {
        'role': 'user',
        'content': 'Dime más sobre el slicing.'
    }
]

response = ollama.chat(
    model='llama3.2',
    messages=mensajes
```

```
)  
  
print(f"Respuesta del Tutor:\n{response['message']['content']}")
```

Diferencias Clave entre Roles:

Rol	Propósito Principal
<code>system</code>	Configurar el comportamiento general ("Eres un pirata", "Eres un traductor").
<code>user</code>	Proporcionar la entrada o el problema a resolver.
<code>assistant</code>	Mantener la continuidad histórica de la conversación.

4.3. Streaming: Optimizando la Latencia Percibida.

En modelos grandes, la generación completa puede tardar varios segundos. El streaming permite recibir palabras según se generan, lo que mejora drásticamente la experiencia de usuario (UX).

```
import ollama  
  
def stream_respuesta(pregunta):  
    print("Respuesta del Asistente: ", end='', flush=True)  
  
    # Activamos stream=True  
    for chunk in ollama.chat(  
        model='llama3.2',  
        messages=[{'role': 'user', 'content': pregunta}],  
        stream=True  
    ):  
        # Cada 'chunk' contiene un fragmento de la respuesta  
        parte_texto = chunk['message']['content']  
        print(parte_texto, end='', flush=True)  
  
stream_respuesta("Escribe un poema corto sobre el hardware de computadoras.")
```

5. Tools y Function Calling

5.1. Tools: Extensión de Capacidades con Funciones Personalizadas.

Las **Tools** (herramientas) representan el puente entre el razonamiento del modelo y el mundo real. A través de este mecanismo, Ollama puede dejar de ser un sistema cerrado para interactuar con bases de datos, APIs externas o el propio sistema de archivos.

Conceptos clave:

- **Esquema de JSON (Input Schema):** El modelo necesita saber con precisión qué datos puede enviarte. Esto se define mediante un esquema que especifica tipos de datos (string, number, boolean) y cuáles son obligatorios.
- **La Alucinación en Tools:** Aunque el modelo es inteligente, a veces puede intentar llamar a funciones que no existen o inventar parámetros. La validación en el lado de Python es obligatoria.
- **Contexto de Herramienta (tool_role):** Cuando devolvemos el resultado de la función al modelo, usamos un rol específico llamado `tool`. Esto le indica al modelo que esa información no la ha dicho un usuario, sino que es un dato verídico obtenido de una ejecución.

¿Por qué usar Tools en lugar de prompts largos?

- **Precisión:** En lugar de pedirle al modelo que "calcule" algo (donde puede fallar matemáticamente), le pedimos que "llame a la función de cálculo".
- **Actualización:** El modelo puede obtener datos en tiempo real (clima, bolsa, stock) que no estaban en su entrenamiento.
- **Acción:** Permite que el modelo ejecute acciones (enviar un correo, apagar una luz, guardar un archivo).

```
import ollama

# 1. Función que simula una consulta a una base de datos
def obtener_precio_producto(id_producto: str):
    base_datos = {"PROD-01": 29.99, "PROD-02": 45.50}
    return base_datos.get(id_producto, "Producto no encontrado")

# 2. Definición profesional de la herramienta
herramientas_tienda = [{
    'type': 'function',
    'function': {
        'name': 'obtener_precio_producto',
        'description': 'Obtiene el precio actual de un producto usando su ID.',
        'parameters': {
            'type': 'object',
            'properties': {
                'id_producto': {
                    'type': 'string',
                    'description': 'El código del producto (ej: PROD-01)',
                },
            },
            'required': ['id_producto'],
        },
    },
}]

# 3. Ejemplo de llamada inicial
pregunta = "¿Cuánto cuesta el producto PROD-02?"
response = ollama.chat(
    model='llama3.1',
```



```

messages=[{'role': 'user', 'content': pregunta}],
tools=herramientas_tienda
)

# El modelo responderá con una solicitud de llamada a la función
if response['message'].get('tool_calls'):
    print("El modelo ha decidido usar una herramienta:")
    for call in response['message']['tool_calls']:
        print(f"- Función: {call['function']['name']}")
        print(f"- Argumentos: {call['function']['arguments']}")

```

5.2. Extracción de Datos de las respuestas de inferencia.

La extracción de datos estructurados es una de las aplicaciones más potentes de Ollama. Permite transformar texto no estructurado (emails, chats, documentos) en formatos que una base de datos o una aplicación pueden procesar directamente, como JSON.

Conceptos clave:

- **Esquema de Salida (Output Schema):** A diferencia de las Tools (donde el esquema es la entrada), aquí el esquema define cómo queremos que el modelo nos devuelva la información.
- **Modo JSON (`format='json'`):** Es una funcionalidad nativa de Ollama que garantiza que la respuesta sea un objeto JSON válido. Sin esto, el modelo podría añadir texto explicativo ("Aquí tienes el JSON...") que rompería los parsers.
- **Instrucciones en el System Prompt:** Para obtener los mejores resultados, el `system prompt` debe ser muy estricto sobre las claves y tipos de datos esperados.

Estrategias para una extracción robusta:

1. **Zero-Shot Extraction:** Pedir directamente el JSON basándose en una descripción.
2. **Constraint Engineering:** Indicar al modelo que **no** incluya explicaciones, solo el bloque de código JSON.
3. **Tipado en Python:** Usar librerías como `Pydantic` (opcionalmente) para validar que el JSON recibido cumple con nuestra estructura antes de usarlo.

```

import ollama
import json

# Texto desordenado de entrada
documento_medico = """
El paciente Juan Pérez, con ID 55432, acude por fiebre de 38.5 grados.
Se le receta Paracetamol 1g cada 8 horas por 5 días.
"""

# Definimos el esquema esperado en el prompt
prompt_instrucciones = f"""
Analiza el siguiente texto y extrae la información en un objeto JSON con esta

```

```

estructura:
{{
  "nombre": "nombre completo",
  "id": "número de identificación",
  "diagnostico": "síntomas principales",
  "tratamiento": {{
    "medicamento": "nombre",
    "dosis": "cantidad",
    "frecuencia": "cada cuantas horas"
  }}
}}
TEXTO A ANALIZAR: {documento_medico}
"""

response = ollama.generate(
    model='llama3.2',
    prompt=prompt_instrucciones,
    format='json', # Crucial para recibir solo JSON puro
    options={'temperature': 0} # Temperatura 0 para máxima fidelidad
)

# Parseo y uso de los datos
datos_limpios = json.loads(response['response'])
print(f"Paciente: {datos_limpios['nombre']}")
print(f"Dosis: {datos_limpios['tratamiento']['dosis']}")

```

5.3. Conceptos de Agentes y herramientas externas.

El "Function Calling" o "Herramientas" (Tools) permite que el modelo rompa las barreras de su entrenamiento estático. En lugar de limitarse a predecir la siguiente palabra, el modelo puede **decidir** que necesita información externa o realizar una acción y delegar la ejecución en tu código Python.

Conceptos clave:

- **Inferencia no es Ejecución:** Es vital entender que el LLM **no ejecuta** el código Python. El modelo solo genera una "solicitud estructurada" (JSON) indicando qué función quiere usar y con qué datos. Es tu entorno de Python el que realiza la acción real.
- **El Bucle del Agente (Agent Loop):** El proceso suele requerir al menos dos llamadas al modelo: una para que el modelo elija la herramienta, y otra para que el modelo redacte la respuesta final basándose en el resultado de esa herramienta.
- **Modelos de Lenguaje como 'Orquestadores':** En este flujo, el LLM actúa como un cerebro que coordina herramientas, no solo como un generador de texto.

¿Cómo funciona realmente el flujo de trabajo?

1. **Definición:** Le pasamos al modelo una lista de funciones con sus nombres, descripciones y esquemas de parámetros (usando JSON Schema).
2. **Razonamiento:** Si la pregunta del usuario lo requiere (ej: "¿Cuál es el saldo de mi cuenta?"), el

modelo devuelve un mensaje especial con los detalles de la `tool_call`.

3. **Ejecución Local:** Tu código intercepta esa solicitud, valida los argumentos, ejecuta la lógica técnica (consultar API, DB, etc.) y obtiene un resultado.
4. **Respuesta Final:** Se envía el resultado de la función de vuelta al modelo (con el rol `tool`) para que lo traduzca a un lenguaje humano comprensible.

```
import ollama

# 0. Definimos una función que el modelo 'conoce' pero no puede ejecutar solo
def calcular_impuesto(monto: float, pais: str) -> float:
    tasas = {'España': 0.21, 'México': 0.16, 'Colombia': 0.19}
    return monto * tasas.get(pais, 0.15)

# 1. Definición de la Tool para el modelo
herramientas = [{
    'type': 'function',
    'function': {
        'name': 'calcular_impuesto',
        'description': 'Calcula el impuesto aplicable a un monto según el país.',
        'parameters': {
            'type': 'object',
            'properties': {
                'monto': {'type': 'number', 'description': 'El importe base.'},
                'pais': {'type': 'string', 'description': 'País para el cálculo.'},
            },
            'required': ['monto', 'pais'],
        },
    },
}]

# 2. El modelo analiza la intención del usuario
response = ollama.chat(
    model='llama3.1',
    messages=[{'role': 'user', 'content': '¿Cuanto pagaré de impuestos por 1000 euros en España?'}],
    tools=herramientas
)

# 3. Verificamos si el modelo solicita la herramienta
if response['message'].get('tool_calls'):
    for call in response['message']['tool_calls']:
        if call['function']['name'] == 'calcular_impuesto':
            # Ejecución real en Python
            args = call['function']['arguments']
            impuesto = calcular_impuesto(args['monto'], args['pais'])
            print(f"Calculado internamente: {impuesto}")
```

5.4. Ejemplo Práctico: Agente de Información Meteorológica.

A continuación, implementamos un ciclo completo y funcional de una Tool. Este patrón es la base para crear agentes inteligentes.

```
import ollama

# 1. Definimos la función real que ejecutará Python
def obtener_clima(ciudad: str) -> str:
    # En un caso real, aquí llamarías a la API de OpenWeather
    # Para el ejemplo, simulamos una respuesta
    datos_clima = {
        "Madrid": "22°C y soleado",
        "Barcelona": "18°C con lluvia",
        "México": "25°C nublado"
    }
    return datos_clima.get(ciudad, "Clima desconocido para esa ubicación")

# 2. Definición de la Tool para Ollama (Esquema JSON)
tools = [{
    'type': 'function',
    'function': {
        'name': 'obtener_clima',
        'description': 'Obtiene la temperatura y estado del tiempo actual de una ciudad.',
        'parameters': {
            'type': 'object',
            'properties': {
                'ciudad': {
                    'type': 'string',
                    'description': 'El nombre de la ciudad, por ejemplo: Madrid, París, Londres',
                },
            },
            'required': ['ciudad'],
        },
    },
}]

# 3. Interacción inicial con el modelo
messages = [{'role': 'user', 'content': '¿Debo llevar paraguas si voy a Barcelona hoy?'}]
print(f"Pregunta: {messages[0]['content']}")

response = ollama.chat(
    model='llama3.1', # Recomendado usar versiones que soporten bien tools
    messages=messages,
    tools=tools,
```

```

)

# 4. Procesamiento de la decisión del modelo (Tool Loop)
if response['message'].get('tool_calls'):
    # El modelo decidió que necesita usar una herramienta
    for call in response['message']['tool_calls']:
        if call['function']['name'] == 'obtener_clima':
            # Extraemos los argumentos propuestos por el modelo
            args = call['function']['arguments']
            print(f"-> El modelo solicita ejecutar 'obtener_clima' para:
{args['ciudad']}")

            # Ejecutamos la lógica real en Python
            resultado = obtener_clima(args['ciudad'])

    # 5. Volvemos al modelo con el resultado para que nos de la respuesta
final
    messages.append(response['message']) # Añadimos la petición del modelo
    messages.append({
        'role': 'tool',
        'name': 'obtener_clima',
        'content': resultado
    })

    final_res = ollama.chat(model='llama3.1', messages=messages)
    print(f"Respuesta final: {final_res['message']['content']}")
else:
    print(f"Respuesta directa (sin herramientas): {response['message']['content']}")

```

5.4.1. Puntos Clave para el Éxito con Tools:

- **Descripciones Claras:** El modelo decide usar la herramienta basándose en la **description**. Sé muy específico con lo que hace la función.
- **Validación de Argumentos:** Siempre valida los datos que propone el modelo (el **args['ciudad']**) antes de pasarlos a tu lógica crítica.
- **Modelos Compatibles:** Asegúrate de usar modelos entrenados específicamente para function calling (como **llama3.1**, **mistral-nemo**, etc.) para mayor fiabilidad.

6. Visión y Modelos Multimodales

6.1. Capacidades de visión (OCR, descripción, análisis).

Modelos como **llava** o **moondream** permiten procesar imágenes. Esto abre puertas a:

- Lectura de facturas o tickets (OCR avanzado).
- Generación de subtítulos para accesibilidad.
- Análisis de capturas de pantalla para QA automático.

```
import ollama

with open('factura.jpg', 'rb') as f:
    img_bytes = f.read()

response = ollama.generate(
    model='llava',
    prompt='Extrae el importe total y la fecha de esta factura en formato JSON.',
    images=[img_bytes],
    format='json' # Forzamos salida estructurada si el modelo lo permite
)
print(response['response'])
```

7. Embeddings y Flujos RAG (Retrieval Augmented Generation)

7.1. Fundamentos de Embeddings: Del Texto al Espacio Vectorial.

Los **Embeddings** son vectores matemáticos que representan conceptos. A diferencia de una búsqueda tradicional por palabras clave (donde "casa" y "hogar" son diferentes), los embeddings permiten que la máquina entienda que ambos términos son semánticamente cercanos.

Conceptos clave:

- **La Metáfora del Mapa:** Imagina un mapa donde las ciudades son palabras. Si dos ciudades están cerca en el mapa, es porque sus significados son parecidos.
- **Normalización:** Los vectores suelen normalizarse para que su longitud sea 1, facilitando el cálculo de la **Similitud del Coseno** (el ángulo entre dos vectores).

```
import ollama
import numpy as np

def calcular_similitud(v1, v2):
    # La similitud del coseno mide el ángulo entre dos vectores
    return np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))

# Generamos vectores para tres frases
v_gato = ollama.embeddings(model='mxbai-embed-large', prompt='El gato duerme en el sofá')['embedding']
v_felino = ollama.embeddings(model='mxbai-embed-large', prompt='Un pequeño felino descansa')['embedding']
v_espacio = ollama.embeddings(model='mxbai-embed-large', prompt='El telescopio James Webb explora galaxias')['embedding']

print(f"Similitud Gato vs Felino: {calcular_similitud(v_gato, v_felino):.4f}")
```

```
print(f"Similitud Gato vs Espacio: {calcular_similitud(v_gato, v_espacio):.4f}")
```

7.2. Arquitectura RAG: Combinar Memoria y Recuperación para Respuestas Actualizadas.

El LLM tiene un '\punto de corte\' en su entrenamiento (ej: no sabe nada de 2025). El RAG funciona como un **examen a libro abierto**: el modelo no necesita memorizarlo todo, solo necesita saber buscar en los apuntes que le proporcionamos.

Los 4 Pilares del RAG:

1. **Fragmentación (Chunking)**: Cortamos documentos largos en trozos de ~500 palabras para no saturar la ventana de contexto.
2. **Base de Datos Vectorial**: Un almacén especializado (como ChromaDB o FAISS) para buscar vectores a gran velocidad.
3. **Recuperación (Retrieval)**: Seleccionamos los 'K' fragmentos más parecidos a la duda del usuario.
4. **Inyección de Contexto**: Construimos un prompt robusto que limite al modelo a responder usando solo esos trozos.

7.3. Ejemplo Funcional: Sistema de Soporte con Datos Propios.

```
import ollama

# Nuestra "Base de Conocimiento" local
manual_vuelo_drones = [
    "CHUNKING_1: Para despegar el dron X-Pro, mantén presionado el botón ROJO 3 segundos.",
    "CHUNKING_2: La altitud máxima legal en zonas urbanas es de 120 metros.",
    "CHUNKING_3: Si la batería baja del 15%, el dron activará el modo 'Regreso a Casa' automáticamente."
]

def buscar_en_manual(pregunta_usuario):
    # 1. Convertimos la pregunta en vector
    v_pregunta = ollama.embeddings(model='mxbai-embed-large',
    prompt=pregunta_usuario)['embedding']

    # 2. (Simulación de búsqueda vectorial rápida)
    # En producción usarías una DB vectorial aquí
    fragmento_ganador = manual_vuelo_drones[0] # Simplificación para el ejemplo
    if "altitud" in pregunta_usuario.lower():
        fragmento_ganador = manual_vuelo_drones[1]
    if "batería" in pregunta_usuario.lower() or "baja" in pregunta_usuario.lower():
        fragmento_ganador = manual_vuelo_drones[2]
```

```

    return fragmento_ganador

def asistente_drones(pregunta):
    contexto = buscar_en_manual(pregunta)

    prompt = f"""Eres un asistente técnico de drones. Responde de forma concisa usando
    SOLO el contexto proporcionado.
    SI EL CONTEXTO NO TIENE LA INFORMACIÓN, di que consulten al fabricante.

    CONTEXTO: {contexto}
    PREGUNTA: {pregunta}"""

    response = ollama.chat(model='llama3.2', messages=[{'role': 'user', 'content':
    prompt}])
    return response['message']['content']

print("Duda 1:", asistente_drones("¿A qué altura puedo volar en la ciudad?"))
print("Duda 2:", asistente_drones("¿Qué pasa si me quedo sin batería?"))

```

7.3.1. Desafíos Avanzados en RAG:

- **Alucinaciones con Contexto:** A veces el modelo ignora el contexto y usa su conocimiento previo. Se soluciona con **System Prompts** más agresivos.
- **Top-K y Re-ranking:** No siempre el fragmento más parecido es el mejor. A veces hay que recuperar 10 trozos y usar otro modelo para ordenarlos por relevancia.

8. Escalabilidad y Asincronía

8.1. Programación no bloqueante con Asyncio.

Imprescindible para aplicaciones web (FastAPI/Flask) para no congelar el servidor al procesar una respuesta larga.

```

import asyncio
from ollama import AsyncClient

async def proceso_paralelo(tema):
    client = AsyncClient()
    response = await client.generate(model='llama3.2', prompt=f'Dame 3 datos sobre
    {tema}')
    print(f"--- Datos sobre {tema} ---\n{response['response']}")

async def main():
    # Lanzamos varias peticiones a la vez
    temas = ['Gravedad', 'Internet', 'Cafeína']
    await asyncio.gather(*(proceso_paralelo(t) for t in temas))

```



```
asyncio.run(main())
```

8.2. Escalado Horizontal: Múltiples Instancias de Ollama.

Cuando una sola máquina no es suficiente para atender la demanda, o queremos separar cargas de trabajo (ej: un servidor para inferencia y otro para embeddings), el escalado horizontal se vuelve necesario. Gracias a que la librería de Python usa `httpx`, podemos gestionar múltiples clientes de forma transparente.

Conceptos clave:

- **Balanceo de Carga Lógico:** No siempre necesitamos un balanceador de hardware (como Nginx). Nuestra propia aplicación Python puede decidir a qué servidor enviar una tarea basándose en la disponibilidad o el tipo de modelo.
- **Aislamiento de Recursos:** Ejecutar Ollama en diferentes máquinas evita que una generación larga de un modelo de 70B bloquee peticiones ligeras de otros usuarios.
- **Configuración de `OLLAMA_HOST`:** Por defecto, Ollama solo escucha en `localhost`. Para el escalado horizontal, es vital configurar `OLLAMA_HOST=0.0.0.0` en los servidores remotos para permitir conexiones externas.

Estrategias de Escalado:

1. **Pool de Clientes:** Mantener una lista de objetos `Client` apuntando a diferentes IPs.
2. **Especialización de Nodos:** Un nodo potente con GPU para chat y varios nodos con CPU para tareas de embeddings de fondo.
3. **Round-Robin manual:** Una lógica simple que reparte las peticiones entre los servidores disponibles.

```
import ollama
from ollama import Client
import random

# Definimos nuestras instancias de Ollama en la red local
nodos_ollama = [
    'http://192.168.1.10:11434',
    'http://192.168.1.11:11434',
    'http://192.168.1.12:11434'
]

def obtener_cliente_aleatorio():
    host_elegido = random.choice(nodos_ollama)
    print(f"Enviando petición a: {host_elegido}")
    return Client(host=host_elegido)

# Ejemplo de uso en un flujo de trabajo
```

```
try:
    cliente = obtener_cliente_aleatorio()
    response = cliente.generate(model='llama3', prompt='¿Cuál es la capital de Francia?')
    print(f"Respuesta: {response['response']}")
except Exception as e:
    print(f"Error conectando al nodo: {e}")
```

Ventajas de este enfoque:

- **Alta Disponibilidad:** Si un servidor cae, los otros pueden seguir operando.
- **Flexibilidad:** Puedes añadir o quitar nodos "en caliente" simplemente actualizando la lista de IPs.
- **Coste Eficiente:** Puedes reutilizar hardware antiguo como nodos secundarios para modelos pequeños.

8.3. Monitorización y Logging en Ollama.

Para aplicaciones en producción, "lanzar y olvidar" no es una opción. Necesitamos visibilidad sobre cuánto tiempo tardan las respuestas, cuántos tokens se generan y si hay errores de conexión. Ollama devuelve metadatos valiosos en cada respuesta que podemos aprovechar.

Conceptos clave:

- **Métricas de Latencia:** No solo importa el tiempo total. Debemos distinguir entre el "Tiempo hasta el primer token" (latencia percibida) y la "Velocidad de generación" (tokens por segundo).
- **Observabilidad en el Cliente:** Dado que Ollama no tiene un panel de control nativo complejo, es nuestra responsabilidad en Python capturar y persistir estos logs.
- **Uso de Metadatos de Finalización:** Campos como `total_duration`, `load_duration` y `eval_count` son oro puro para entender si nuestro hardware está saturado.

Qué debemos monitorizar:

- **Load Duration:** Tiempo que tarda el modelo en cargarse de disco a VRAM. Si es muy alto, quizás el modelo se está "descargando" demasiado pronto debido al `keep_alive`.
- **Eval Rate:** Tokens por segundo. Vital para garantizar una buena experiencia de usuario.
- **Errores de HTTP:** Controlar timeouts y códigos 500 para detectar caídas del servidor.

```
import ollama
import time
import logging

# Configuración básica de logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

def inferencia_con_logging(prompt):
    try:
```

```

start_wall_time = time.time()

response = ollama.generate(model='llama3', prompt=prompt)

# Extraemos métricas de Ollama (vienen en nanosegundos)
total_ms = response['total_duration'] / 1e6
load_ms = response['load_duration'] / 1e6
tokens = response.get('eval_count', 0)
tps = (tokens / (response['eval_duration'] / 1e9)) if
response.get('eval_duration') else 0

logging.info(f"Éxito: {tokens} tokens generados en {total_ms:.2f}ms")
logging.info(f"Métricas detalladas: Carga: {load_ms:.2f}ms | Velocidad:
{tps:.2f} tokens/s")

return response['response']

except Exception as e:
    logging.error(f"Error en la petición: {e}")
    return None

# Prueba del sistema
resultado = inferencia_con_logging("¿Cuál es el origen de la palabra 'robot'?")

```

Estrategias Avanzadas:

- **Exportación a Prometheus/Grafana:** Podrías crear un endpoint en Python que exponga estas métricas para ser visualizadas en dashboards profesionales.
- **Alertas de Degradación:** Si la velocidad (tokens/s) cae por debajo de un umbral, disparar una alerta para revisar el estado de la GPU.

9. Mejores Prácticas y Rendimiento

9.1. Manejo de Errores y Robustez del Sistema.

No asumas que el modelo siempre responderá perfecto o que el servidor estará disponible. En entornos de producción, es crítico manejar las excepciones para evitar caídas del sistema.

```

import ollama
import httpx

try:
    # 1. Error de modelo inexistente
    res = ollama.chat(model='modelo_desconocido', messages=[])
except ollama.ResponseError as e:
    if e.status_code == 404:
        print(f"Error 404: El modelo no está descargado. Intentando descargar...")
        # Lógica de auto-pull opcional

```

```

        # ollama.pull('llama3')
    else:
        print(f"Error de API ({e.status_code}): {e.error}")

except httpx.ConnectError:
    # 2. Error de conexión (Ollama no está corriendo)
    print("Error Crítico: No se pudo conectar con el servidor Ollama.")
    print("Asegúrate de que 'ollama serve' esté activo.")

except httpx.ReadTimeout:
    # 3. Timeout en respuestas extremadamente largas o hardware lento
    print("Error: El servidor tardó demasiado en responder.")

finally:
    print("Limpieza de recursos o log de fin de petición.")

```

9.1.1. Validaciones Recomendadas:

- **Verificación previa:** Antes de iniciar una aplicación pesada, utiliza `ollama.list()` para comprobar si los modelos necesarios están instalados.
- **Circuit Breaker:** Si detectas múltiples `ConnectError`, pausa las peticiones al servidor para no colapsar el hilo principal.
- **Sanitización de entrada:** Aunque Ollama maneja bien los prompts, limita el tamaño del texto de entrada para evitar ataques de denegación de servicio (DoS) por consumo excesivo de contexto.

9.2. Optimización de Recursos y Cuantización

9.2.1. ¿Qué es la Cuantización?

La cuantización es el proceso de reducir la precisión de los pesos de un modelo (normalmente de 16 bits a 4 u 8 bits).

Esto es fundamental para:

- **Reducir el uso de VRAM:** Permite ejecutar modelos de 7B-8B parámetros en tarjetas gráficas domésticas de 6GB u 8GB.
- **Aumentar la velocidad (Tokens/s):** Menos datos que mover entre la memoria y el procesador se traduce en una respuesta más rápida.

9.2.2. Formatos comunes en Ollama:

- **Q4_K_M (Recomendado):** El equilibrio perfecto. Ofrece una pérdida de precisión casi imperceptible con una reducción de tamaño del 70%.
- **Q8_0:** Muy alta precisión, casi idéntica al modelo original, pero requiere el doble de memoria que Q4.
- **FP16:** El modelo sin comprimir. Solo usar si se dispone de hardware profesional (A100, H100) y

se requiere máxima exactitud.

9.2.3. Ejemplo Práctico: Perfilado de Rendimiento.

Podemos medir cuánto tarda un modelo según su tamaño para optimizar nuestra aplicación:

```
import ollama
import time

def test_performance(model_name):
    start_time = time.time()
    response = ollama.generate(
        model=model_name,
        prompt="Explica la teoría de la relatividad en tres frases."
    )
    duration = time.time() - start_time
    tokens = response.get('eval_count', 0)

    print(f"\n--- Test Modelo: {model_name} ---")
    print(f"Tiempo total: {duration:.2f}s")
    print(f"Velocidad: {tokens/duration:.2f} tokens/s")
    print(f"VRAM Estimada: {response.get('model_info', {}).get('general.architecture',
'N/A')}}")

# Comparativa entre versiones (si las tienes descargadas)
# test_performance('llama3:8b-instruct-q4_K_M')
# test_performance('llama3:8b-instruct-fp16')
```

9.2.4. Gestión de la Memoria (VRAM):

Ollama gestiona la memoria automáticamente, pero podemos forzar el "descargado" de un modelo para liberar recursos mediante la API:

```
import ollama

# Opción para liberar VRAM inmediatamente poniendo el keep_alive a 0
ollama.generate(model='llama3', prompt='', keep_alive=0)
print("Memoria VRAM liberada correctamente.")
```

10. Proyecto Final: Agente de Soporte Técnico Autónomo

Desarrollo de un sistema que:

1. Recibe un ticket por voz o texto.
2. Usa **Vision** para analizar capturas de pantalla del error.

3. Usa **Tools** para buscar el código de error en una base de datos local.
4. Responde mediante **Streaming** con pasos de solución detallados.